

Advanced Internals — A Complete Learning Book

*Deep-dives into Consumer Groups, Offsets, Exactly-Once Semantics,
Log Compaction, Schema Registry, and Stream Processing —
with intuition, diagrams, code examples, and real-world scenarios.*

Ch 1	Consumer Groups & Partition Rebalancing
---------	---

Ch 2	Offset Management Deep Dive
---------	-----------------------------

Ch 3	Exactly-Once Semantics (EOS)
---------	------------------------------

Ch 4	Log Retention & Compaction
---------	----------------------------

Ch 5	Schema Registry & Avro
---------	------------------------

Ch 6	Kafka Streams & Stream Processing
---------	-----------------------------------

A Professional Guide for Engineers & Data Scientists

Chapter 1

Consumer Groups & Partition Rebalancing

How Kafka automatically distributes work — and what happens when consumers join or leave

■ The Intuition

Imagine you run a pizza restaurant and 100 orders arrive on a conveyor belt. You have three chefs. You naturally divide the belt into three sections — chef A takes orders 1-33, chef B takes 34-66, chef C takes 67-100. If chef B calls in sick, you redistribute the belt between A and C. That is **exactly** how Kafka consumer groups work.

A **Consumer Group** is a set of consumers that cooperate to consume a topic. Kafka assigns each partition to exactly one consumer in the group at any moment. This gives you parallel processing AND guarantees ordering within a partition.

■ Core Concepts

Partition	The unit of parallelism. A topic is split into <u>N partitions</u> .
Consumer Group	One or more consumers sharing a group.id.
Group Coordinator	A Kafka broker that manages membership and assignments.
Group Leader	The first consumer to join; runs the assignment algorithm.
Rebalance	The protocol where partitions are redistributed among consumers.

The Golden Rule: Max Useful Consumers = Partition Count

If your topic has 4 partitions and you have 6 consumers in a group, only 4 consumers get work. The remaining 2 sit idle — they cost resources but process nothing. Conversely, if you have 2 consumers and 4 partitions, each consumer handles 2 partitions.

SCENARIO	PARTITIONS	CONSUMERS	RESULT
Balanced	4	4	Each consumer → 1 partition ■
Under-scaled	4	2	Each consumer → 2 partitions (more load)
Over-scaled	4	6	4 active + 2 idle consumers ■■

How Rebalancing Works — Step by Step

1. A consumer joins or leaves the group.
2. The Group Coordinator detects the change and signals a rebalance.
3. All consumers revoke their current partition assignments.
4. The Group Leader re-runs the assignment algorithm.
5. New assignments are distributed; consumers resume from committed offsets.

■ Why Rebalances Are Painful

During a rebalance, ALL consumption stops. This is called a 'stop-the-world' event.

For high-throughput systems, even a 2-3 second pause can cause downstream latency spikes.

Kafka 2.4+ introduced 'cooperative rebalancing' (Incremental Rebalance) which only revokes the partitions that need to move — dramatically reducing downtime.

■ Examples

Example 1: Basic Consumer Group Setup

Two consumers sharing a 4-partition topic:

Python — confluent-kafka

```
from confluent_kafka import Consumer

# Consumer A — same group.id means they cooperate
consumer_a = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'order-processing-group',
    'auto.offset.reset': 'earliest'
})

# Consumer B — identical config, different process/thread
consumer_b = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'order-processing-group', # <-- same group!
    'auto.offset.reset': 'earliest'
```

```

})

# Kafka automatically assigns partitions:
# consumer_a -> [partition-0, partition-1]
# consumer_b -> [partition-2, partition-3]
consumer_a.subscribe(['orders'])
consumer_b.subscribe(['orders'])

```

Example 2: Detecting Partition Assignment via Callbacks

Python — handling rebalance callbacks

```

def on_assign(consumer, partitions):
    print(f'Assigned: {[p.partition for p in partitions]}')
def on_revoke(consumer, partitions):
    print(f'Revoked: {[p.partition for p in partitions]}')
# IMPORTANT: commit offsets before losing partitions
consumer.commit(asynchronous=False)
consumer.subscribe(['orders'], on_assign=on_assign, on_revoke=on_revoke)
while True:
    msg = consumer.poll(timeout=1.0)
    if msg and not msg.error():
        process(msg)

```

Example 3: Cooperative (Incremental) Rebalancing — Kafka 2.4+

Use CooperativeStickyAssignor to minimise pause time during rebalances:

Python — cooperative rebalancing

```

consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'ml-inference-group',
    # Cooperative rebalance: only move partitions that MUST move
    'partition.assignment.strategy': 'cooperative-sticky',
})

```

Example 4: Checking Group Lag with kafka-consumer-groups CLI

CLI — inspecting consumer group lag

```

# See which partitions each consumer owns and how far behind they are
kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--group order-processing-group \
--describe

# Output columns:
# TOPIC PARTITION CURRENT-OFFSET LOG-END-OFFSET LAG CONSUMER-ID

```

```
# orders 0 5432 5432 0 consumer-1-abc
# orders 1 5410 5450 40 consumer-1-abc
# orders 2 5432 5432 0 consumer-2-def
# orders 3 5432 5432 0 consumer-2-def
# LAG = LOG-END-OFFSET - CURRENT-OFFSET
# Partition 1 is 40 messages behind – needs investigation!
```

Example 5: Scaling Out — Adding a Consumer Dynamically

Conceptual — dynamic scaling

```
# System starts with 2 consumers for a 4-partition topic
# Each consumer owns 2 partitions
# You deploy a 3rd consumer (e.g., traffic spike):
# Kafka triggers a rebalance automatically
# New assignment after rebalance:
# consumer-1 -> [partition-0, partition-1] (unchanged)
# consumer-2 -> [partition-2] (lost one)
# consumer-3 -> [partition-3] (new arrival)
# Scale back down – consumer-3 crashes:
# Kafka detects missed heartbeats (session.timeout.ms = 30s default)
# Triggers rebalance, consumer-2 takes back partition-3
```

■ Best Practices

Set `session.timeout.ms` (default 30s) lower for faster failure detection, but not too low or you'll get false rebalances.

Use cooperative-sticky partitioner in production to reduce rebalance impact.

Match your consumer count to partition count for optimal throughput.

Chapter 2

Offset Management Deep Dive

Sync vs async commit, manual offset control, and replaying historical data

■ The Intuition

Think of an offset like a bookmark in a book. **Kafka stores messages forever (until retention kicks in).** The offset tells Kafka: 'This consumer has read up to page 342.' If the consumer crashes and restarts, it opens to page 343 — not page 1.

The critical question is: **when** do you move the bookmark? **Move it too early and a crash means lost messages. Move it too late and a crash means duplicate processing. This trade-off defines the three delivery guarantees.**

■ Core Concepts

Offset Storage

Offsets are stored in a special internal Kafka topic called `consumer.offsets`. Each entry is keyed by (group.id, topic, partition) and maps to the next offset to be consumed.

Mode	When Committed	Risk	Use Case
Auto Commit (<code>enable.auto.commit=true</code>)	Every N ms in background	Crash between poll → duplicate	Quick prototypes
Sync Commit (<code>commit()</code>)	After each batch, blocks until ACK	Slow — waits for broker	Critical pipelines
Async Commit (<code>commit(asynchronous=True)</code>)	Non-blocking, fire-and-forget	Out-of-order commits possible	High-throughput
Manual Specific Offset (<code>commit(offsets={...})</code>)	You choose exactly when	More code complexity	Fine-grained control

■ Examples

Example 1: Auto Commit — Simple but Risky

Python — auto commit

```

consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'demo-group',
    'enable.auto.commit': True, # default: True
    'auto.commit.interval.ms': 5000, # commit every 5 seconds
})

# Risk: if process() takes 6 seconds and crashes at 4s,
# the offset was NOT committed yet – messages reprocessed on restart.

while True:
    msg = consumer.poll(1.0)
    if msg:
        process(msg) # if this crashes, offset may NOT be committed

```

Example 2: Synchronous Manual Commit

Python — synchronous commit

```

consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'safe-group',
    'enable.auto.commit': False, # we control commits
})
consumer.subscribe(['orders'])

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None:
        continue
    if msg.error():
        continue

    # Step 1: Process first
    process_order(msg.value())

    # Step 2: THEN commit (at-least-once guarantee)
    consumer.commit(asynchronous=False) # blocks until broker confirms
    # If commit fails, an exception is raised – handle it!

```

Example 3: Async Commit with Error Callback

Python — async commit with callback

```

def commit_callback(err, partitions):
    if err:
        print(f'Commit failed: {err}')
        # Decide: retry? alert? dead-letter queue?
    else:
        for p in partitions:
            print(f'Committed {p.topic}[{p.partition}] @ {p.offset}')

```

```

while True:
    msg = consumer.poll(1.0)
    if msg and not msg.error():
        process(msg)
    # Non-blocking - returns immediately
    consumer.commit(asynchronous=True, callback=commit_callback)

```

Example 4: Committing Specific Offsets (Batch Processing)

Python — batch commit specific offsets

```

from confluent_kafka import TopicPartition

batch = []
while len(batch) < 100: # collect 100 messages
    msg = consumer.poll(1.0)
    if msg and not msg.error():
        batch.append(msg)
    # Process entire batch (e.g., bulk insert to database)
    bulk_insert_to_db([m.value() for m in batch])
    # Commit only the highest offset per partition
    offsets_to_commit = {}
    for msg in batch:
        key = (msg.topic(), msg.partition())
        # offset+1 means 'next message to read'
        offsets_to_commit[key] = TopicPartition(
            msg.topic(), msg.partition(), msg.offset() + 1
        )
    consumer.commit(offsets=list(offsets_to_commit.values()), asynchronous=False)

```

Example 5: Seeking Backward — Replay Messages After ML Bug

Your ML model had a bug for 2 hours. You need to reprocess those messages:

Python — timestamp-based offset seeking

```

from confluent_kafka import TopicPartition
from datetime import datetime, timezone, timedelta

# Find the offset corresponding to 2 hours ago
two_hours_ago = datetime.now(timezone.utc) - timedelta(hours=2)
timestamp_ms = int(two_hours_ago.timestamp() * 1000)

# Ask Kafka: what was the offset at that timestamp?
partitions = consumer.assignment()
ts_partitions = [TopicPartition(p.topic, p.partition, timestamp_ms)
                 for p in partitions]

offsets_for_times = consumer.offsets_for_times(ts_partitions)

# Seek each partition back to that point

```

```
for tp in offsets_for_times:
    if tp.offset >= 0:
        consumer.seek(tp)
        print(f'Rewound {tp.topic}[{tp.partition}] to offset {tp.offset}')
        # Now poll – you'll replay from 2 hours ago
        while True:
            msg = consumer.poll(1.0)
            reprocess_with_fixed_model(msg)
```

■ At-Least-Once Recipe (Recommended for ML Pipelines)

Disable auto commit: `enable.auto.commit = False`

Process the message completely (write to DB, call model, etc.)

Then commit the offset.

On crash and restart, the last un-committed message is reprocessed — make your processing idempotent (safe to run twice).

Chapter 3

Exactly-Once Semantics (EOS)

Making consume + transform + produce a single atomic operation

■ The Intuition

Imagine a bank transfer: debit account A, credit account B. If the system crashes between the two steps, you've lost money or created money — neither is acceptable. You need both to happen together or not at all. That is **atomicity**, and Kafka's EOS provides exactly that.

The Three Delivery Guarantees

Guarantee	Mechanism	Risk	Best For
At-Most-Once	Commit offset BEFORE processing.	Messages can be lost on crash.	Metrics, logs where loss is OK
At-Least-Once	Commit offset AFTER processing.	Messages may be processed twice on crash.	Most ML pipelines — make processing idempotent
Exactly-Once	Kafka transaction: consume + produce + commit are atomic.	No duplicates, no loss.	Financial, order processing, billing

How Kafka Transactions Work

Kafka uses a **two-phase commit protocol** under the hood. A transactional producer gets a unique `transactional.id`. It sends messages with a transaction marker. When it calls `commit_transaction()`, a special COMMIT marker is appended. Consumers configured with `isolation.level=read_committed` only see messages from completed transactions.

■ Examples

Example 1: Idempotent Producer — Prevent Duplicate Messages

The simplest EOS upgrade: enable idempotence on the producer. Kafka assigns a Producer ID (PID) and sequence number to each message. If a network hiccup causes a retry, Kafka deduplicates automatically.

Python — idempotent producer

```
from confluent_kafka import Producer

producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'enable.idempotence': True, # prevents duplicates on retry
    # These are auto-set when idempotence is enabled:
    # acks = 'all'
    # max.in.flight.requests.per.connection = 5
    # retries = INT_MAX
})

producer.produce('payments', key='tx-001', value='{"amount": 100}')
producer.flush() # Guaranteed exactly-once delivery to Kafka
```

Example 2: Full Transactional Producer

Python — transactional producer

```
producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'transactional.id': 'payment-processor-1', # must be unique per instance
    'enable.idempotence': True,
})

producer.init_transactions() # registers with broker, gets epoch
try:
    producer.begin_transaction()
    producer.produce('processed-payments', value=process(raw_event))
    producer.produce('audit-log', value=make_audit(raw_event))
    producer.commit_transaction() # both messages visible atomically
except Exception as e:
    producer.abort_transaction() # neither message becomes visible
    raise
```

Example 3: Consume-Transform-Produce (The EOS Pattern)

Python — full EOS consume-transform-produce

```
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'eos-group',
    'enable.auto.commit': False, # we commit inside the transaction
    'isolation.level': 'read_committed', # ignore uncommitted messages
})

producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'transactional.id': 'eos-processor-1',
    'enable.idempotence': True,
```

```

})
producer.init_transactions()
while True:
    msg = consumer.poll(1.0)
    if not msg or msg.error(): continue
    producer.begin_transaction()
    # Transform and produce output
    result = transform(msg.value())
    producer.produce('output-topic', value=result)
    # Send consumer offset AS PART of the transaction
    producer.send_offsets_to_transaction(
        [{msg.topic(): {msg.partition(): msg.offset() + 1}}],
        consumer.consumer_group_metadata()
    )
    producer.commit_transaction()
    # Atomically: output produced + offset committed

```

Example 4: Reading Only Committed Messages (Consumer Side)

Python — `read_committed` isolation level

```

consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'downstream-group',
    # Only see messages from COMMITTED transactions
    'isolation.level': 'read_committed',
    # Default is 'read_uncommitted' – sees all messages including
    # those from aborted transactions!
})
# Messages from aborted transactions are filtered out automatically.
# This is how downstream services stay clean.

```

Example 5: EOS for a Billing Pipeline

Real-world scenario: charge a customer and emit an invoice, atomically:

Python — billing pipeline with EOS

```

producer.init_transactions()
for raw_event in consumer_poll_loop():
    try:
        producer.begin_transaction()
        # Compute charge
        charge = compute_charge(raw_event)
        # Write to two topics – both or neither
        producer.produce('charges', value=charge.to_json())
    except:
        producer.abort_transaction()

```

```
producer.produce('invoices', value=make_invoice(charge))
# Advance the consumer offset atomically
producer.send_offsets_to_transaction(
    get_offsets(raw_event),
    consumer.consumer_group_metadata()
)
producer.commit_transaction()
except KafkaException:
    producer.abort_transaction()
# retry or send to dead-letter topic
```

■ EOS Performance Cost

EOS adds ~10-20% latency overhead due to the 2-phase commit protocol.

Only use it where correctness matters more than raw throughput.

For ML feature pipelines, at-least-once + idempotent processing is usually sufficient.

Chapter 4

Log Retention & Compaction

How Kafka decides which messages to keep — and log compaction for state topics

■ The Intuition

Kafka is a distributed log. By default it keeps everything — but disks are finite. Kafka gives you two policies to manage this:

Time/Size Retention: Delete messages older than N days (or bigger than N GB). Think of it as a rolling newspaper archive — keep the last 7 days.

Log Compaction: Keep only the *latest* message for each key. Think of it as a contacts list — you only care about the current phone number, not every edit you ever made.

■ Retention Policies Compared

Policy	Keeps	Deletes	Best For
Time Retention (log.retention.hours)	All messages within window	Messages older than N hours/days	Event logs, clickstream, IoT sensor data
Size Retention (log.retention.bytes)	Messages up to N bytes per partition	Oldest messages when size limit hit	Fixed-disk environments
Log Compaction (clean up.policy=compact)	Latest value per key	Older values for the same key	User state, config, CDC (change data capture)
Compact + Delete (both policies)	Latest value per key within window	Compacted old values + expired messages	State topics with eventual cleanup

How Log Compaction Works Internally

Kafka's log cleaner runs in the background. It reads segments, builds a map of key → latest offset, then rewrites segments keeping only the latest value per key. The result: the log always contains the full current state, but old history is pruned.

To delete a key entirely, produce a **tombstone** — a message with the key and a null value. The compactor will eventually remove it.

■ Examples

Example 1: Creating a Compacted Topic (CLI)

CLI — create compacted topic

```
# Create a compacted topic for user state
kafka-topics.sh \
--bootstrap-server localhost:9092 \
--create \
--topic user_state \
--partitions 6 \
--replication-factor 3 \
--config cleanup.policy=compact \
--config min.cleanable.dirty.ratio=0.1 \
--config segment.ms=60000

# cleanup.policy=compact → enable compaction
# min.cleanable.dirty.ratio=0.1 → compact when 10% of log is dirty
# segment.ms=60000 → roll segment every 60s (helps compaction)
```

Example 2: Writing State to a Compacted Topic

Python — writing and deleting from compacted topic

```
from confluent_kafka import Producer
import json

producer = Producer({'bootstrap.servers': 'localhost:9092'})

# Update user state – key is user_id
def update_user_state(user_id, state_dict):
    producer.produce(
        topic='user_state',
        key=str(user_id),
        value=json.dumps(state_dict).encode(),
    )
    producer.flush()

update_user_state('user-123', {'plan': 'premium', 'region': 'us-east'})
update_user_state('user-123', {'plan': 'free', 'region': 'us-east'})
# After compaction, only the second message survives for user-123

# Tombstone: delete user entirely
producer.produce('user_state', key='user-123', value=None) # null value
producer.flush()

# After compaction, user-123 is gone
```

Example 3: Time-Based Retention Config

CLI + Python — time-based retention

```
# Keep messages for 7 days
kafka-configs.sh \
--bootstrap-server localhost:9092 \
--entity-type topics \
--entity-name clickstream \
--alter \
--add-config retention.ms=604800000 # 7 days in ms

# Or in Python when creating a topic:
from confluent_kafka.admin import AdminClient, NewTopic
admin = AdminClient({'bootstrap.servers': 'localhost:9092'})
topic = NewTopic('clickstream', num_partitions=12, replication_factor=3,
config={'retention.ms': '604800000'})
admin.create_topics([topic])
```

Example 4: Reading a Compacted Topic as a State Store

Python — rebuilding state from compacted topic

```
# On startup, read the ENTIRE compacted topic to rebuild state
from confluent_kafka import Consumer, TopicPartition, OFFSET_BEGINNING
consumer = Consumer({'bootstrap.servers': 'localhost:9092',
'group.id': 'state-builder',
'auto.offset.reset': 'earliest'})

# Get all partitions
metadata = consumer.list_topics('user_state')
partitions = [TopicPartition('user_state', p)
for p in metadata.topics['user_state'].partitions]
consumer.assign(partitions)

# Seek to beginning and read until caught up
state_store = {}
end_offsets = consumer.get_watermark_offsets
for tp in partitions:
consumer.seek(TopicPartition(tp.topic, tp.partition, OFFSET_BEGINNING))

while True:
msg = consumer.poll(0.5)
if msg is None: break # caught up
if msg.value() is None:
state_store.pop(msg.key(), None) # tombstone = delete
else:
state_store[msg.key()] = json.loads(msg.value())
```

Example 5: Compact + Delete Policy (Best of Both Worlds)

CLI — compact+delete policy

```
# Keep latest value per key, but also expire old entries after 30 days
```

```
kafka-topics.sh \  
--bootstrap-server localhost:9092 \  
--create \  
--topic user_events_state \  
--config cleanup.policy=compact,delete \  
--config retention.ms=2592000000 \ # 30 days  
--config delete.retention.ms=86400000 # tombstones kept for 24h  
  
# Result: always see latest state per user,  
# but users inactive for 30+ days are eventually evicted.
```

■ When to Use Compaction vs Retention

Use TIME retention for: event streams (clicks, sensor data, logs) where older events naturally lose value.

Use COMPACTION for: state topics (user profiles, product catalog, config) where you care about current state, not history.

Use BOTH for: CDC (change-data-capture) streams where you need current state AND automatic cleanup of stale keys.

Chapter 5

Schema Registry & Avro

Enforcing message contracts so a renamed field never silently breaks a pipeline

■ The Intuition

Imagine two teams: Team A produces order events, Team B consumes them. Team A renames 'amount' to 'total_amount' in a Friday deploy. Team B's code crashes Monday morning processing the weekend's orders. Without a schema registry, this is silent until production breaks.

A **Schema Registry** is a central catalog of message schemas. Before a producer sends a message, it registers the schema. The schema ID is embedded in every message. Consumers look up the schema by ID to deserialize. Incompatible schema changes are **rejected at the registry** — before they reach production.

■ Avro Data Types

Apache Avro is the most common serialization format used with Schema Registry. It is compact (binary), fast, and self-describing. Schemas are defined in JSON.

JSON — Avro schema definition

```
# A typical Avro schema for an Order event
{
  "namespace": "com.mycompany.orders",
  "type": "record",
  "name": "Order",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "customer_id", "type": "string"},
    {"name": "amount", "type": "double"},
    {"name": "status", "type": {"type": "enum",
      "name": "Status",
      "symbols": ["PENDING", "PAID", "SHIPPED"]}},
    {"name": "created_at", "type": "long",
      "logicalType": "timestamp-millis"},
    {"name": "metadata", "type": ["null", "string"], "default": null}
  ]
}
```

Schema Compatibility Modes

Mode	Allows	Rejects
BACKWARD (default)	Add optional field with default Remove optional field	Add required field (no default) Change field type
FORWARD	Remove optional field Add optional field	Add required field Change field type
FULL	Add/remove optional fields with defaults	Any breaking change
NONE	Everything	Nothing (dangerous)

■ Examples

Example 1: Installing and Running Confluent Schema Registry

Docker — running Schema Registry

```
# docker-compose.yml snippet
services:
  schema-registry:
    image: confluentinc/cp-schema-registry:7.5.0
    environment:
      SCHEMA_REGISTRY_HOST_NAME: schema-registry
      SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:9092
    ports:
      - '8081:8081'

# Verify it's running:
curl http://localhost:8081/subjects

# Returns: [] (empty schema list initially)
```

Example 2: Producing with Avro and Schema Registry

Python — Avro producer with Schema Registry

```
from confluent_kafka import Producer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import SerializationContext, MessageField

schema_str = '''
{ "type": "record", "name": "Order",
  "fields": [{"name": "order_id", "type": "string"},
{"name": "amount", "type": "double"}] }
'''
```

```

sr_client = SchemaRegistryClient({'url': 'http://localhost:8081'})
avro_serializer = AvroSerializer(sr_client, schema_str)
producer = Producer({'bootstrap.servers': 'localhost:9092'})
order = {'order_id': 'ord-001', 'amount': 99.99}
producer.produce(
    topic='orders',
    value=avro_serializer(order, SerializationContext('orders', MessageField.VALUE)),
)
producer.flush()
print('Order sent with Avro schema ID embedded in message')

```

Example 3: Consuming with Avro Deserializer

Python — Avro consumer

```

from confluent_kafka import Consumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer
from confluent_kafka.serialization import SerializationContext, MessageField

sr_client = SchemaRegistryClient({'url': 'http://localhost:8081'})
avro_deserializer = AvroDeserializer(sr_client)

consumer = Consumer({'bootstrap.servers': 'localhost:9092',
                    'group.id': 'order-consumer-group',
                    'auto.offset.reset': 'earliest'})
consumer.subscribe(['orders'])

while True:
    msg = consumer.poll(1.0)
    if msg is None: continue
    order = avro_deserializer(
        msg.value(),
        SerializationContext('orders', MessageField.VALUE)
    )
    print(f'Order: {order}')
# Prints: Order: {'order_id': 'ord-001', 'amount': 99.99}

```

Example 4: Backward-Compatible Schema Evolution

Add an optional field with a default — existing consumers still work:

JSON — schema evolution example

```

# Original schema (version 1):
{ "fields": [{"name": "order_id", "type": "string"},
{"name": "amount", "type": "double"}] }

# Evolved schema (version 2) — BACKWARD COMPATIBLE:
# Added 'currency' with a default value
{ "fields": [{"name": "order_id", "type": "string"},

```

```
{"name": "amount", "type": "double"},
{"name": "currency", "type": "string", "default": "USD"}] }

# Registry accepts this ■ (old consumers read new messages, get 'USD')
# INCOMPATIBLE change – Registry REJECTS this ■:
# Renaming 'amount' to 'total_amount' with no default
# Old consumers can't find 'amount' field and crash
```

Example 5: Checking and Managing Schemas via REST API

CLI — Schema Registry REST API

```
# List all registered subjects
curl http://localhost:8081/subjects

# Get all versions of a subject
curl http://localhost:8081/subjects/orders-value/versions

# Returns: [1, 2, 3]

# Get schema version 2
curl http://localhost:8081/subjects/orders-value/versions/2

# Test if a schema is compatible before registering
curl -X POST
http://localhost:8081/compatibility/subjects/orders-value/versions/latest \
-H 'Content-Type: application/json' \
-d '{"schema": "{...new schema JSON...}"}'

# Returns: {"is_compatible": true} or {"is_compatible": false}

# Set compatibility level for a subject
curl -X PUT http://localhost:8081/config/orders-value \
-H 'Content-Type: application/json' \
-d '{"compatibility": "BACKWARD}"'
```

■ Schema Registry Best Practices

Always use BACKWARD compatibility (the default) — it means new code can read old messages.

Never rename a field — add a new field with a default instead.

Run schema compatibility checks in your CI/CD pipeline before any deploy.

Use subject naming strategy 'TopicNameStrategy' (default): each topic has one schema.

Chapter 6

Kafka Streams & Stream Processing with Faust

Replacing fragile stateful scripts with fault-tolerant stream processors

■ The Intuition

Your manual `transformer.py` script reads from Kafka, maintains a Python dict for state, and produces results. It works fine — until it crashes. On restart, the dict is empty. State is lost. You start replaying messages but aren't sure where you left off.

Stream processing frameworks like **Faust** (Python-native) solve this by storing state in **RocksDB** (an embedded key-value store), using Kafka topics as a changelog for state recovery, automatically managing offsets, and distributing work across multiple workers.

■ Faust vs Manual Script

Feature	Manual <code>transformer.py</code>	Faust
State storage	Python dict (lost on crash)	RocksDB (persistent)
Fault tolerance	Manual restart logic needed	Automatic recovery
Offset management	Manual commit code	Automatic
Scaling	Run multiple scripts, coordinate manually	Add workers, auto-rebalance
Windowing	DIY time-window logic	Built-in tumbling/hopping windows
Joins	Very hard to implement correctly	Built-in stream-table joins

■ Examples

Example 1: Hello Faust — Basic Stream Processor

Python — Faust basic agent

```
# pip install faust-streaming
import faust

app = faust.App(
    'order-processor',
    broker='kafka://localhost:9092',
    value_serializer='json',
)

# Define message schema
class Order(faust.Record):
    order_id: str
    customer_id: str
    amount: float

# Define input and output topics
orders_topic = app.topic('orders', value_type=Order)
processed_topic = app.topic('processed', value_type=Order)

@app.agent(orders_topic)
async def process_orders(orders):
    async for order in orders:
        # Transform
        order.amount = round(order.amount * 1.1, 2) # add 10% tax
        await processed_topic.send(value=order)
        print(f'Processed order {order.order_id}')

# Run: faust -A myapp worker -l info
```

Example 2: Stateful Stream Processing — Per-Customer Running Total

Python — stateful Faust table (persistent state)

```
import faust

app = faust.App('revenue-tracker', broker='kafka://localhost:9092')

class Order(faust.Record):
    customer_id: str
    amount: float

orders_topic = app.topic('orders', value_type=Order)

# State table — persisted in RocksDB, replicated via Kafka changelog
# Survives worker restarts automatically!
revenue_table = app.Table('customer_revenue', default=float)

@app.agent(orders_topic)
async def track_revenue(orders):
    async for order in orders:
        # Increment state — this persists across restarts
        revenue_table[order.customer_id] += order.amount
        print(f'Customer {order.customer_id}: '
              f'total = {revenue_table[order.customer_id]:.2f}')

# If the worker crashes and restarts:
```

```
# 1. Faust reads the RocksDB state from disk
# 2. Replays any missed changelog entries from Kafka
# 3. Continues from exactly where it left off
```

Example 3: Tumbling Window — Orders Per Minute

Python — windowed stream processing

```
from datetime import timedelta
import faust

app = faust.App('windowed-counts', broker='kafka://localhost:9092')

class Order(faust.Record):
    order_id: str
    amount: float

orders_topic = app.topic('orders', value_type=Order)

# Tumbling window: count orders in non-overlapping 1-minute buckets
order_counts = app.Table(
    'order-counts',
    default=int,
).tumbling(timedelta(minutes=1), expires=timedelta(hours=1))

@app.agent(orders_topic)
async def count_orders(orders):
    async for order in orders.group_by(Order.order_id):
        order_counts['total'] += 1
    # Access current window value
    current = order_counts['total'].current()
    print(f'Orders in this minute: {current}')
```

Example 4: Stream-Table Join — Enriching Events with User Data

Python — stream-table join for enrichment

```
import faust

app = faust.App('enricher', broker='kafka://localhost:9092')

class Order(faust.Record):
    order_id: str
    customer_id: str
    amount: float

class User(faust.Record):
    customer_id: str
    name: str
    tier: str # 'gold', 'silver', 'bronze'

orders_topic = app.topic('orders', value_type=Order)
users_topic = app.topic('users', value_type=User)

# Read user data from a compacted topic into a local table
users_table = app.GlobalTable('users', value_type=User)
```

```

@app.agent(orders_topic)
async def enrich_orders(orders):
    async for order in orders:
        user = users_table.get(order.customer_id)
        if user:
            discount = 0.1 if user.tier == 'gold' else 0.0
            print(f'{user.name} ({user.tier}): {order.amount * (1-discount):.2f}')
        else:
            print(f'Unknown customer: {order.customer_id}')

```

Example 5: Replacing transformer.py — Full Fault-Tolerant Version

Python — full migration from transformer.py to Faust

```

"""
Before: transformer.py – fragile, stateful, no recovery
state = {} # lost on crash!
while True:
    msg = consumer.poll()
    state[msg.key] = update(state.get(msg.key, {}), msg.value)
    producer.produce('output', state[msg.key])
    consumer.commit() # manual, easy to forget
After: Faust – fault-tolerant, scalable, self-recovering
"""

import faust
app = faust.App(
    'state-transformer',
    broker='kafka://localhost:9092',
    # Store state in RocksDB (default)
    store='rocksdb://',
)

class Event(faust.Record): key: str; value: dict
input_topic = app.topic('input', value_type=Event)
output_topic = app.topic('output', value_type=Event)

# State survives crashes – stored in RocksDB + Kafka changelog
state_table = app.Table('state', default=dict)

@app.agent(input_topic)
async def transform(events):
    async for event in events:
        # Merge new data into existing state
        current = state_table[event.key]
        current.update(event.value)
        state_table[event.key] = current
        await output_topic.send(value=Event(key=event.key, value=current))

# Offsets committed automatically by Faust

```

```
# Scale: faust -A myapp worker (run on 3 machines for 3x throughput)
```

■ Faust vs Kafka Streams (Java)

Faust is a Python-native implementation inspired by Kafka Streams (Java).

Choose Faust if your team is Python-first — same concepts, Python syntax.

Choose Kafka Streams (Java) for higher throughput or JVM ecosystem integration.

Both use RocksDB for local state and Kafka for changelog replication.

■ Running Faust in Production

Run multiple workers: `faust -A myapp worker -l info` (on each node)

Faust partitions state tables across workers automatically.

Use `--without-web` to disable the built-in monitoring HTTP server if not needed.

Monitor with: `faust -A myapp stats`